

bok25

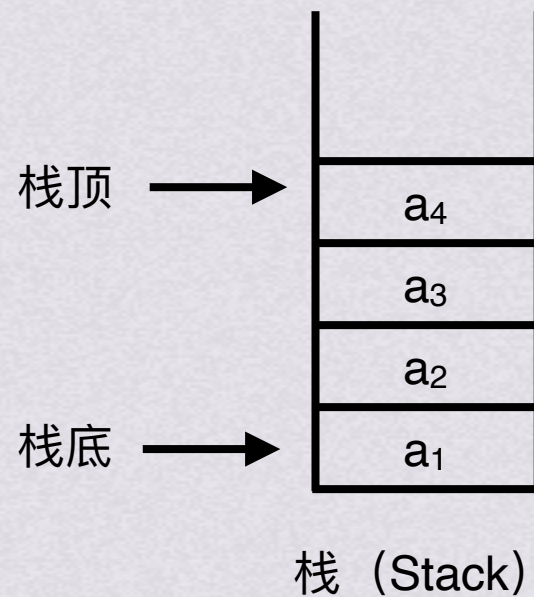
基
礎
班

、考研数据結構



栈是什么？

只能在一端（表尾）进行插入和删除的线性表





只能在一端（表尾）进行插入和删除的线性表

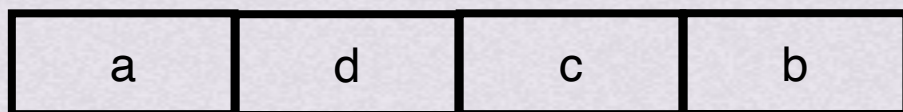
假设一种操作序列：进 出 进 进 进 出 出 出



栈 (Stack)

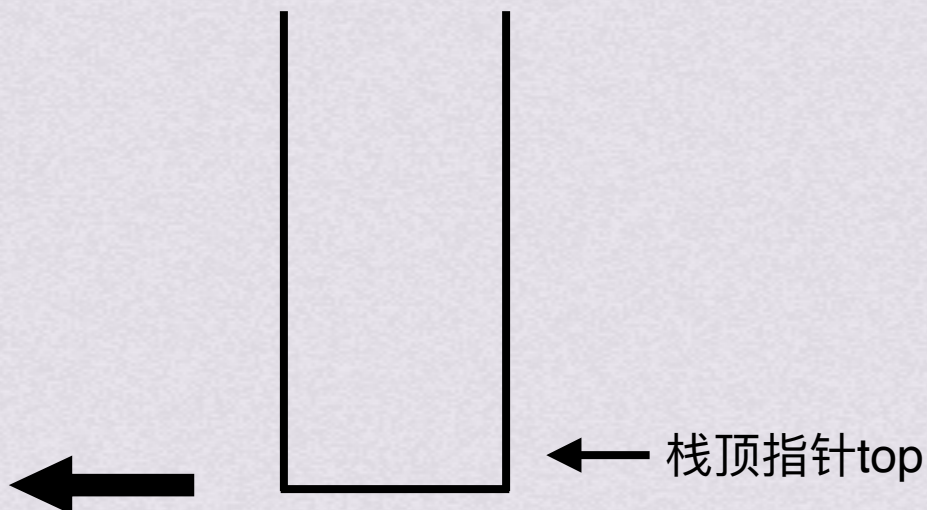


只能在一端（表尾）进行插入和删除的线性表



假设一种操作序列：进 出 进 进 进 出 出 出

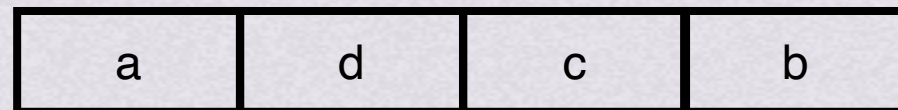
push(a) //a入栈
pop //a出栈
push(b) //b入栈
push(c) //c入栈
push(d) //d入栈
pop //d出栈
pop //c出栈
pop //b出栈



栈 (Stack)



只能在一端（表尾）进行插入和删除的线性表



最终得到的序列

对于同一个入栈序列，如果我们规定的操作序列不同，那么出栈序列也会不同。所以，在没有限定操作序列的前提下，入栈序列和出栈序列会呈现一对多的关系



基本操作

ADT Stack{

数据对象 : $D = \{a_i \mid a_i \in \text{ElemSet}, i = 1, 2, \dots, n, n \geq 0\}$

数据关系 : $R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i = 2, \dots, n \}$

约定 a_n 端为栈顶, a_1 端为栈底。

基本操作 :

InitStack(&S)

操作结果 : 构造一个空栈S。

DestroyStack(&S)

初始条件 : 栈S已存在。

操作结果 : 栈S被销毁。

ClearStack(&S)

初始条件 : 栈S已存在。

操作结果 : 将S清为空栈。

StackEmpty(S)

初始条件 : 栈S已存在。

操作结果 : 若栈S为空栈, 则返回true, 否则返回false。

StackLength(S)

初始条件 : 栈S已存在。

操作结果 : 返回S的元素个数, 即栈的长度。

GetTop(S)

初始条件 : 栈S已存在且非空。

操作结果 : 返回S的栈顶元素, 不修改栈顶指针。

Push(&S, e)

初始条件 : 栈S已存在。

操作结果 : 插入元素e为新的栈顶元素。

Pop(&S, &e)

初始条件 : 栈S已存在且非空。

操作结果 : 删除S的栈顶元素, 并用e返回其值。

StackTraverse(S)

初始条件 : 栈S已存在且非空。

操作结果 : 从栈底到栈顶依次对S的每个数据元素进行访问。

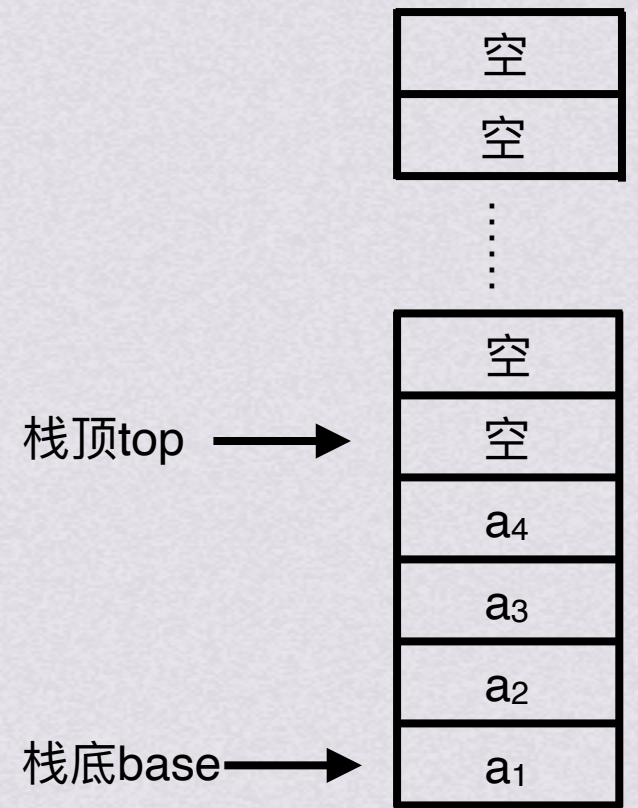
}ADT Stack



顺序栈

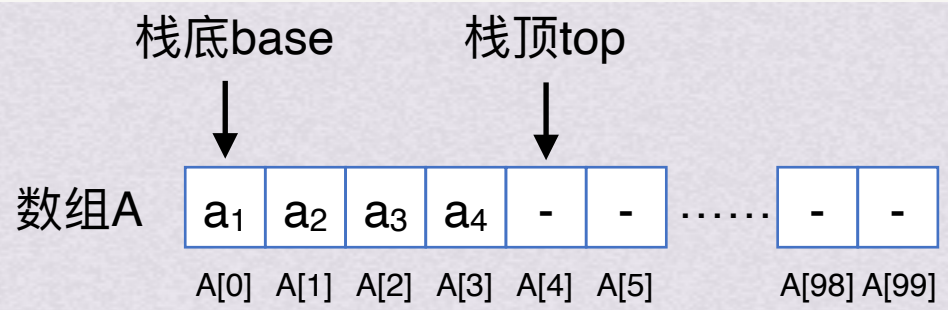
```
//- - - - - 顺序栈的存储结构- - - - -  
#define MAXSIZE 100    //顺序栈存储空间的初始分配量  
typedef struct  
{  
    SElemType *base;    //栈底指针  
    SElemType *top;      //栈顶指针  
    int stacksize;      //栈可用的最大容量  
}SqStack ;
```

最大可用容量：100

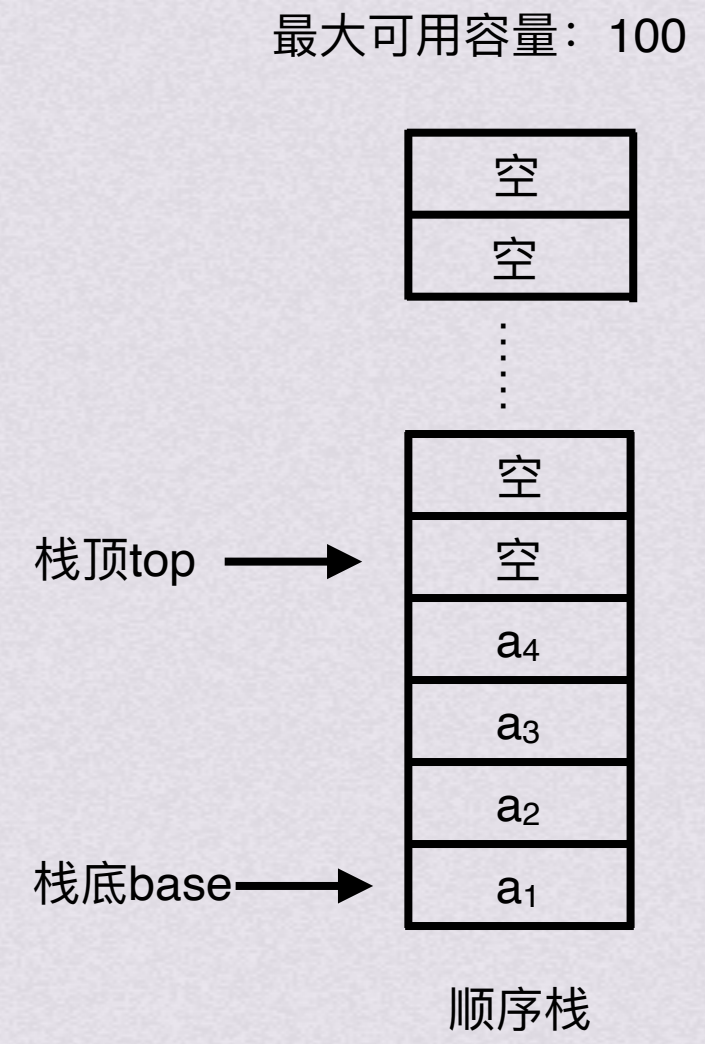


顺序栈

```
//- - - - - 顺序栈的存储结构- - - - -
#define MAXSIZE 100 //顺序栈存储空间的初始分配量
typedef struct
{
    SElemType *base; //栈底指针
    SElemType *top; //栈顶指针
    int stacksize; //栈可用的最大容量
}SqStack ;
```



对应存储该顺序栈的数组情况





顺序栈

初始化操作

```
Status InitStack(SqStack &S)
{
    // 构造一个空栈 S
    S.base = new SElemType[MAXSIZE]; // 为顺序栈动态分配一个最大容量为MAXSIZE的数组空间
    if (!S.base) exit(OVERFLOW);    // 存储分配失败
    S.top = S.base;                  // top初始为base, 空栈
    S.stacksize = MAXSIZE;           // stacksize置为栈的最大容量MAXSIZE
    return OK;
}
```



顺序栈

入栈操作

```
Status Push(SqStack &S, SElemType e)
{
    // 插入元素 e 为新的栈顶元素
    if(S.top - S.base == S.stacksize) return ERROR ; // 栈满
    *S.top++ = e; // 将元素e压入栈顶，栈顶指针加1
    return OK ;
}
```



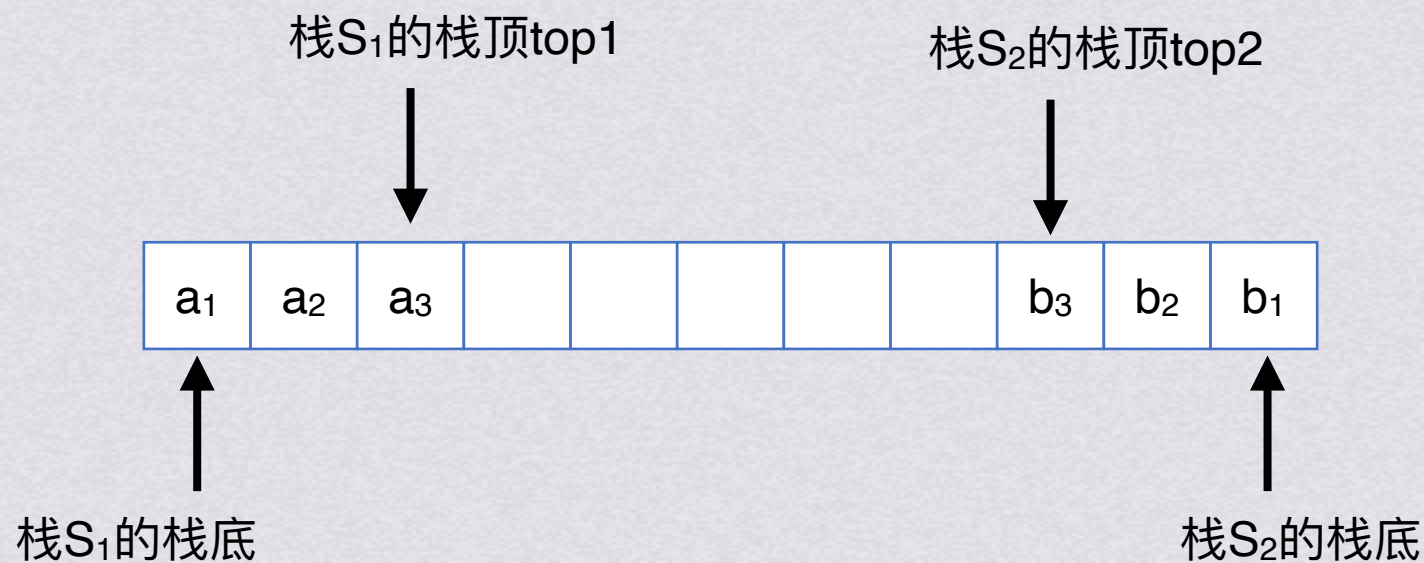

顺序栈

取栈顶元素

```
SElemType GetTop(SqStack S)
{
    // 返回 S 的栈顶元素，不修改栈顶指针
    if(S.top != S.base) // 栈非空
        return *(S.top - 1); // 返回栈顶元素的值，栈顶指针不变
}
```




共享栈



栈顶指针指向栈顶元素

top=-1时栈空

top₂-top₁=1时共享栈满

S₁入栈时, $*++top_1=e;$

S₂入栈时, $*--top_2=e;$



链栈

存储结构

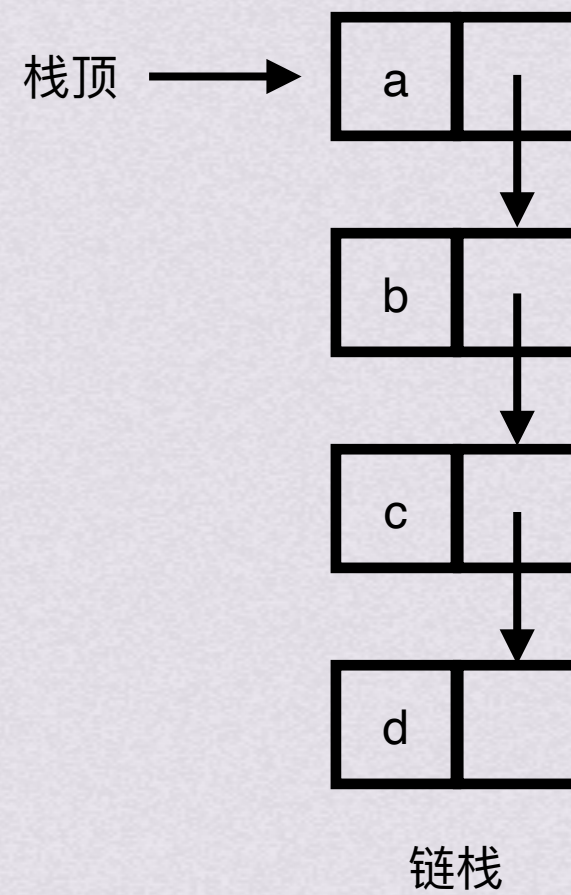
```
//- - - - - 链栈的存储结构- - - - -  
typedef struct StackNode {  
    ElemType data;           //数据部分  
    struct StackNode *next;  //指针部分  
}StackNode, *LinkStack;
```




链栈

存储结构

```
//----- 链栈的存储结构-----  
typedef struct StackNode {  
    ElemType data;           //数据部分  
    struct StackNode *next;  //指针部分  
}StackNode, *LinkStack;
```





链栈

初始化

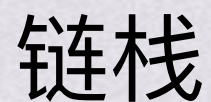
```
Status InitStack(LinkStack &S) {  
    //构造一个空栈 S，栈顶指针置空  
    S=NULL;  
    return OK;  
}
```




链栈

入栈

```
Status Push(LinkStack &S, SElemType e)
{
    // 在栈顶插入元素 e
    p=new StackNode; //生成新节点
    p->data=e;        //将新节点数据域置为e
    p->next=S;        //将新节点插入栈顶
    S=p;              //修改栈顶指针为p
    return OK;
}
```



出栈

[illegible]

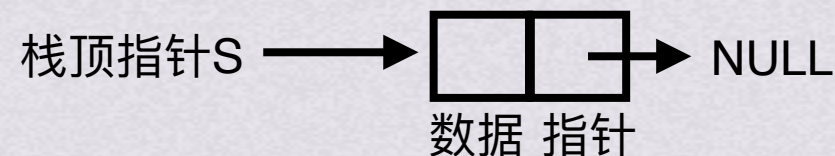


链栈出栈步骤演示

```
Status Pop(LinkStack &S, SElemType &e) {  
    // 删除 S 的栈顶元素, 用 e 返回其值  
    if(S==NULL) return ERROR;           // 栈空  
    e=S->data;                           // 将栈顶元素赋给e  
    p=S;                                 // 用p临时保存栈顶元素空间, 以备释放  
    S=S->next;                           // 修改栈顶指针  
    delete p;                            // 释放原栈顶元素的空间  
    return OK;  
}
```

- 1.判断栈是否为空，若空则返回ERROR
- 2.将栈顶元素赋给e
- 3.临时保存栈顶元素的空间，以备释放
- 4.修改栈顶指针，指向新的栈顶元素
- 5.释放原栈顶元素的空间。

e= data





链栈

取栈顶元素

```
SElemType GetTop(LinkStack S)
{ // 返回 S 的栈顶元素，不修改栈顶指针
    if (S != NULL) // 栈非空
        return S->data; // 返回栈顶元素的值，栈顶指针不变
}
```




栈的应用

1. 括号匹配
2. 表达式求值
3. 数制转换
4. 递归



栈的应用

1. 括号匹配

- 括号匹配需要解决的问题：
1. 左括号必须用相同类型的右括号闭合
 2. 左括号必须以正确的顺序闭合
 3. 每个右括号都有一个对应的相同类型的左括号

解决思路：每次出现了一类左括号后，计算机都等待和它相同类型的右括号出现。但是在这个过程中会出现不同类型的括号，所以下一次如果出现新类型的括号，我们会把它视为更高优先级的括号，计算机转变为等待和新类型相同的右括号出现。如果出现了所等待的和左括号类型相同的右括号，则两两消除



栈的应用

2.表达式求值

两个数字中间夹着运算符

中缀表达式： $(3+4) \times 5 - 6$

- 需要学习的内容：
- 1.逆波兰表达式求值（后缀表达式）
 - 2.前缀表达式求值
 - 3.中缀表达式转逆波兰式（后缀表达式）



栈的应用

2.表达式求值 2-1.逆波兰表达式求值（后缀表达式）

（中缀表达式） $5 + 8$  $58+$ （后缀表达式）

逆波兰表达式求值过程：

- 1.遇到操作数即压栈
- 2.遇到操作符，即将栈顶的两个元素取出进行计算，并将结果压栈



栈的应用

2.表达式求值

2-1.逆波兰表达式求值（后缀表达式）

逆波兰表达式求值过程：

- 1.遇到操作数即压栈
- 2.遇到操作符，即将栈顶的两个元素取出进行计算，并将结果压栈

中缀表达式： $(3+4) \times 5 - 6$

逆波兰表达式（后缀表达式）： $3\ 4\ +\ 5\ \times\ 6\ -$



栈的应用

2.表达式求值 2-2.前缀表达式求值

给出中缀表达式: $[(1+2) \times 3] + [(4 / (6-2)) \times 5]$

试试转前缀表达式!

前缀表达式: $+ \times + 1 2 3 \times / 4 - 6 2 5$



栈的应用

2.表达式求值 2-2.前缀表达式求值

前缀表达式求值的过程：

前缀表达式：

$+ \times + 1 \ 2 \ 3 \times / 4 - 6 \ 2 \ 5$

1.遇到操作数即压栈

2.遇到操作符，即将栈顶的两个元素取出进行计算，并将结果压栈

但此时的扫描顺序会变成从后往前



栈的应用2.表达式求值

2-3.中缀表达式转逆波兰式（后缀表达式）

1.建立一个空栈

给出中缀表达式 $(1+2) \times 3 + (4 / (6-2)) \times 5$

2.开始扫描表达式

求后缀表达式

如果遇到操作数，直接输出，不做栈操作

如果遇到运算符：

依次弹出栈中优先级大于等于当前运算符的所有运算符并输出，碰到左括号“（”或栈空停止。

当前运算符入栈

如果遇到界限符：

遇到左括号“（”，入栈

遇到右括号“）”，依次弹出栈中运算符并输出，

直到左扩号”（“弹出为止

按上述方法处理完所有字符后，将栈中
剩余运算符依次弹出并加入后缀表达式



栈的应用3.数制转换

目的：对于输入的任意一个非负 x 进制整数，打印与其等值的 y 进制数

- 方法：
1. 对于一个十进制数 N 需要转换为 d 进制数，计算 $N \% d$ ，将结果压栈
 2. 计算 $N = N / d$ ，重复上一步骤，直至 N 为0
 3. 元素依次出栈，即为等值的数制转换结果



栈的应用3.数制转换

目的：对于输入的任意一个非负 x 进制整数，打印与其等值的 y 进制数

方法：1.对于一个十进制数 N 需要转换为 d 进制数，计算 $N\%d$ ，将结果压栈

2.计算 $N=N/d$ ，重复上一步骤，直至 N 为0

3.元素依次出栈，即为等值的数制转换结果

将十进制数1348转八进制数

$$\begin{array}{r} 8 \overline{) 1348} \\ 8 \overline{) 168} \quad 4 \\ 8 \overline{) 21} \quad 0 \\ 8 \overline{) 2} \quad 5 \\ \quad 0 \quad 2 \end{array}$$

八进制结果：



计算用栈



栈的应用3.数制转换

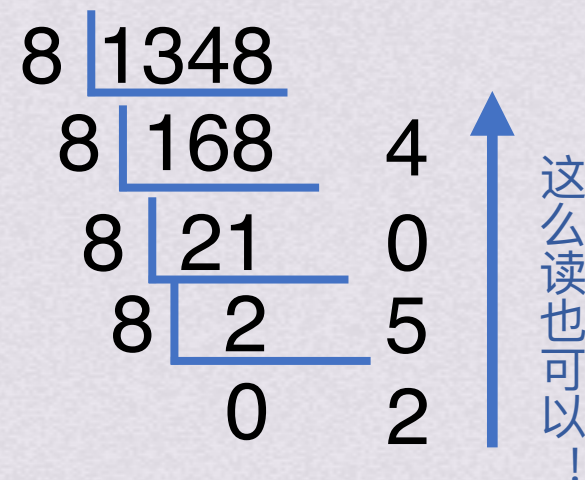
目的：对于输入的任意一个非负 x 进制整数，打印与其等值的 y 进制数

方法：1.对于一个十进制数 N 需要转换为 d 进制数，计算 $N\%d$ ，将结果压栈

2.计算 $N=N/d$ ，重复上一步骤，直至 N 为0

3.元素依次出栈，即为等值的数制转换结果

将十进制数1348转八进制数



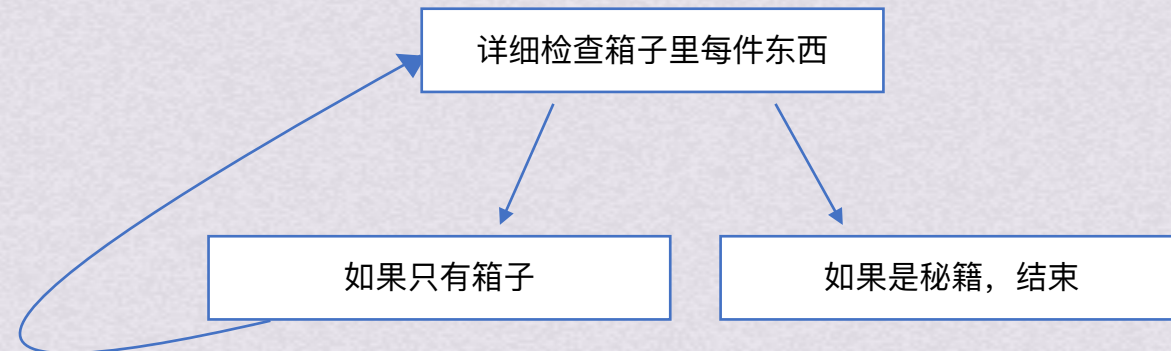
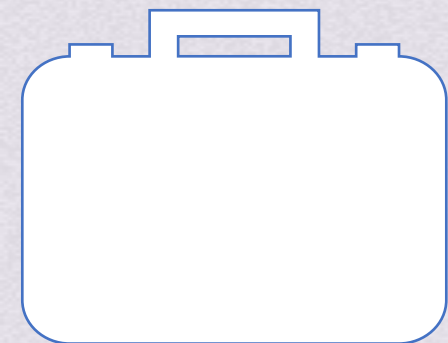
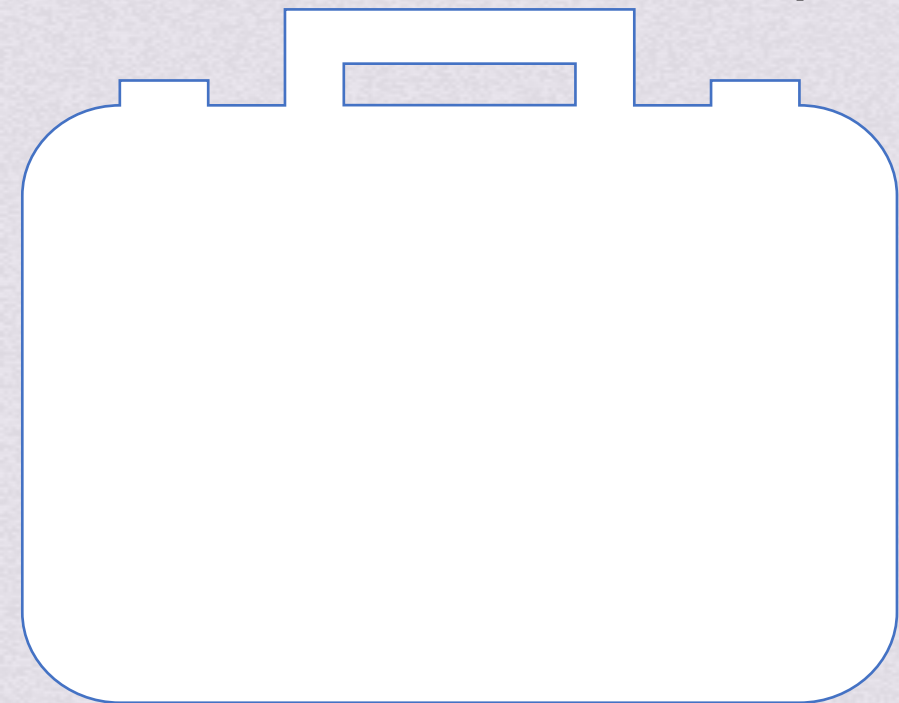
八进制结果：2 5 0 4

计算用栈





栈的应用3.递归中的应用





栈的应用3.递归中的应用

```
int fact(x){  
    if(x == 1)  
        return 1;  
    else return x * fact(x-1);  
}
```



卡特兰数

有n个元素入栈，那么有多少种不同的出栈方式？

答案：卡特兰数

$$\frac{1}{n+1} C_{2n}^n$$

$$\text{其中 } C_{2n}^n = \frac{A_{2n}^n}{(n!)}$$

$$\text{例如, } C_6^3 = \frac{6*5*4}{3*2*1}$$

$$C_8^4 = \frac{8*7*6*5}{4*3*2*1}$$